FEM-Solver-Integration in VisualopenEMS

Planungsvergleich: AWS Palace (awslabs/palace) gegen EdgeFEM (jman4162/EdgeFEM) als zweiter Solver neben openEMS/FDTD

Planungsstudie — nur Planung, keine Implementierung
Stand: 8. November 2026
Projekt: VisualopenEMS — github.com/istra711/VisualOpenEMS
Basis: VisualopenEMS v0.10.0 (434 Tests grün)

Kurzfazit: Beide Solver sind grundsätzlich als zweites Backend neben dem FDTD-Solver integrierbar. Für VisualopenEMS ist **EdgeFEM das bessere Integrationsobjekt** (Python-SDK, Touchstone/VTK-Exporte, kein WSL, Windows nur als Wheel-Aufgabe) — **Palace der bessere Solver** (Reife, MPI/GPU, Eigenmoden, WavePorts), aber mit Windows/WSL-Betreibsmodell. Empfehlung: EdgeFEM als erstes FEM-Backend, Palace später als schweres Backend; Phase 0 (Solver-Abstraktion) und Phase 1 (Geometrie-zu-gmsh-Meshpipeline) solver-neutral vorziehen.

Inhalt

1. Zusammenfassung
2. Ausgangslage: Architektur von VisualopenEMS
3. Kandidat A: AWS Palace
4. Kandidat B: EdgeFEM
5. Direktvergleich
6. Empfehlung
7. Integrationsarchitektur (solver-neutral)
8. Phasenplan
9. Risiken und Mitigationen
10. Offene Entscheidungen
11. Quellen

1. Zusammenfassung

Diese Studie prüft zwei Open-Source-3D-FEM-Elektromagneticsolver darauf, ob sie als zweiter Solver (neben dem vorhandenen openEMS/FDTD-Solver) in VisualopenEMS integrierbar sind. Bewusst wurde nur geplant; es wurden keine Codeänderungen vorgenommen.

Kernaussagen

- **Beide sind integrierbar** — an derselben Nahtstelle: heute *generate_run_script()* + *SimulationRunner* plus formatbasierte Ergebnis-Viewer.
- **Architektur-Fit:** EdgeFEM ist eine Python-Bibliothek (pybind11) und passt in das bestehende *project_run.py*-Muster; Palace läuft als externes MPI-Binary und benötigt auf Windows WSL oder Docker.
- **Ergebnis-Fit:** EdgeFEM exportiert direkt Touchstone (.sNp) und VTK (.vtu) — genau die Formate, die die Viewer von VisualopenEMS bereits lesen; Palace liefert CSV, die adaptert werden müssen.
- **Physik:** EdgeFEM kann dispersiven Material (Debye/Lorentz/Drude), aber keinen TEM-/Mikrostriplen-Port und rechnet erst stabil ab etwa 1 GHz; Palace kann keine dispersiven Materialien, aber Eigenmoden, WavePorts mit Dielektrikum-Querschnitt und große Probleme (MPI, AMR).
- **Reife:** Palace = AWS-Produkt (600 Sterne, dichte Doku, HPC); EdgeFEM = 7 Sterne, Einzelautor, v1.0, aber mit CI-Benchmarks und ehrlicher Limitationstabelle.
- **Größte gemeinsame Investition:** solver-neutrale Backend-Abstraktion (Phase 0) und eine Geometrie-zu-gmsh-Volumenmesh-Pipeline (Phase 1) — beide läufen für jeden der beiden Kandidaten gleichermaßen.

Empfehlung in einem Satz

EdgeFEM als erstes FEM-Backend (MVP), Palace als späteres schweres Backend — vorausgesetzt, die Abstraktion und die Meshpipeline werden von Anfang an solver-neutral gebaut.

2. Ausgangslage: Architektur von VisualopenEMS

Die Einbautstellen wurden durch eine Code-Analyse des Projekts ermittelt:

2.1 Vorhandene Nahtstellen (Seams)

```
generate_run_script(project, ...)  ->  work//project_run.py  (openEMS in-process)
SimulationRunner.start(script, cb) ->  Popen(python -u script), stdout-Stream, abort
EngineProgress.feed(line)          ->  openEMS-spezifisches Log-Regex
Postproc sparams/vtk/nf2ff         ->  lesergestuetzt (Formate, nicht Engines)
```

- Skriptgenerierung: *app/simulation/script_gen.py* (SCRIPT_TEMPLATE, offene EMS-Aufrufe, Material-/Port-/Boundary-Mapping).
- Prozesslauf: *app/simulation/runner.py* (Popen, Log-Drain, Terminate).
- Fortschritt: *app/simulation/progress.py* (parst Timestep-/Energy-Zeilen).
- Ergebnisse: *app/postproc/sparams.py* (sparams.json + Touchstone via skrf), *vtk_export.py* (.vtr), *nf2ff.py* (nf2ff.json).
- Mehrports: *app/simulation/multirun.py* (plan_port_runs / merge_port_runs) — Spaltenweise S-Matrix, für FEM direkt wiederverwendbar.
- **Es gibt keine Solver-Abstraktion**; openEMS/CSXCAD sind an vielen Stellen hart verdrahtet. Die einzigen sauberen Schnittstellen sind Skript-Generator und Runner.

2.2 Projektmodell und Ports

- Projektdatei *.fcsim.json*: unbekannte Keys werden beim Laden stillschweigend verworfen, fehlende bekommen Defaults → ein neuer Top-Level-Key *solver* ist rückwärtskompatibel.
- *PortDef* speichert bereits R, L und C — L und C werden an openEMS though nie übergeben; für FEM (LumpedPort mit R/L/C) sind sie sofort nutzbar.

- Porttypen heute: lumped, msl, rect_waveguide, waveguide, probe.
- Grenzen (BoundarySpec): PEC/PMC/MUR/PML — FDTD-spezifisch.
- Meshing: easyMesh, kartesisches FDTD-Gitter (*app/meshing/*) — für FEM nicht brauchbar; es existiert bereits ein neutrales Grid-Export-Präzedens (*app/meshing/grid_export.py*).
- Einstellungen: *app/settings.py* hält openems_root, simulation_python, freecad_cmd — ein Feld *palace_launcher* bzw. *edgefem*-Pfad wäre die natürliche Ergänzung.

3. Kandidat A: AWS Palace

3.1 Profil

- 3D-FEM-Vollwellenlöser (MFEM + libCEED), Frequenz- und Zeitbereich, Apache-2.0, github.com/awslabs/palace, rund 600 Sterne, AWS Center for Quantum Computing.
- Aufruf: *palace -np <N> config.json* (MPI-Wrapper), Config = JSON mit den fünf Abschnitten Problem / Model / Domains / Boundaries / Solver.
- Validierung per JSON-Schema (*validate-config*), *--dry-run* liefert die aufgelöste Config ohne Rechnung — ideal für Unit-Tests ohne installiertes Palace.
- Problemtypen: Driven (S-Params, uniform oder adaptiver Fast Sweep), Eigenmode (inkl. EPR/Q), Transient, Electrostatic/Magnetostatic (C-/M-Matrizen), BoundaryMode (2D-Wellenleitermoden).
- Ports: LumpedPort (R und L/C, Multielement), WavePort (numerischer 2D-Modensolve, auch mit Dielektrikum-Querschnitt), FloquetPort, SurfaceCurrent.
- Materialien: linear und frequenzunabhängig (εr, μr, tanδ, σ) — **keine dispersiven Modelle**.
- Randbedingungen: PEC, PMC, Impedance, Absorbing (1./2. Ordnung), periodisch; **kein PML** (laut Doku „coming soon“).
- Grenzen: kein offizielles Windows (nur WSL „best-effort“, kein Binary), Versionen untereinander nicht kompatibel (Version hart pinnen), S-Matrix braucht einen Lauf pro Portspalte (oder Multi-Excitation).

3.2 Ergebnis-Dateien (Adapterarbeit)

Palace-Output	Ziel in VisualopenEMS	Viewer
port-S.csv (dB + Phase je Anregung)	sparams.json + Touchstone .s1p/.s2p	sparam_viewer — unverändert
paraview/ (VTK-XML, .vtu/.pvtu)	VTK-Serie im Work-Verzeichnis	vtk_viewer — Reader prüfen
farfield-rE.csv	nf2ff.json-Adapter (θ/φ → Gain)	antenna_viewer — unverändert
eig.csv	neue kleine Eigenmoden-Ansicht	neu (kein FDTD-Äquivalent)
palace.json / stdout	eigener FemProgress-Parser	Statusleiste

3.3 Integrationsaufwand

- JSON-Config-Generator aus primitives/materials/ports/boundary/mesh (neu).
- WSL-/Docker-Runner mit Pfad-Mapping, MPI-Lifecycle, Abort — das größte Betriebsrisiko für eine Windows-PyInstaller-App.
- CSV-Adapter, eigener Fortschritts-Parser, Multirun-Spaltenläufe.
- Gesamteinschätzung: **hoch** (siehe Phasenplan).

4. Kandidat B: EdgeFEM

4.1 Profil

- 3D-FEM für RF/mmWave, Nédélec-Kantelemente, MIT-Lizenz, github.com/jman4162/EdgeFEM, 119 Commits, 7 Sterne, Einzelautor, v1.0.

- C++20-Kern + pybind11-Python-SDK: *pip install edgefem* — **in-process nutzbar**, genau wie heute openEMS im generierten Skript.
- Meshing: gmsh als Abhängigkeit (*load_gmsh*), Eingabe nur Gmsh-msh2 (ASCII).
- Ports: Lumped (Antennen-Feed) und WavePort — WavePort aber nur skalar, homogene Hohlleiter; **„Inhomogeneous / TEM ports (coax, microstrip): not supported“**.
- Materialien: **dispersiv (Debye, Lorentz, Drude, DrudeLorentz)** — die einzige der beiden Optionen mit dieser Fähigkeit.
- Randbedingungen: 1. Ordnung ABC plus Verlustschicht; echtes PML erst v1.1 (geplant).
- Exporte: **Touchstone .sNp, VTK .vtu, CSV** — passen direkt zu den Viewern.
- Weitere Fähigkeiten: Stratton-Chu-Farfield, periodische Bloch-Randbedingung (eine Achse), Analyse-Design-Klassen (RectWaveguideDesign, PatchAntennaDesign, UnitCellDesign), 14 Plotfunktionen.
- Validierung: WR-90-Benchmarks 5–26 GHz in CI (|S21| 0.997–0.9997, Passivität ≤ 1.05), dokumentierte Meshkonvergenz.

4.2 Grenzen (selbsterklärt im README)

- **Kein TEM/Mikrostriplen-Port** — für VisualopenEMSs MSL-Ports nur Lumped-Approximation am Port-Ende möglich.
- Frequenzbereich: Low-Frequency-Stabilisierung nicht implementiert — Probleme unter etwa 1 GHz; HF/VHF bleibt beim FDTD-Solver.
- Nur Frequenzbereich (kein Time-Domain), single-threaded, Direktlöser, klein/mittel (etwa bis 1 M DOF), kein AMR, höhere Elemente erst v1.1.
- Installation: PyPI-Wheel nur für macOS arm64; Windows baut aus sdist (cmake, Eigen, C++20) — für Endnutzer müssen **wir** ein Windows-Wheel bauen (GitHub Actions).
- Reife-Risiko: 7 Sterne, Bus-Factor 1, ambitionierte Roadmap (PML, Tet10, OpenMP, MPI in v2.0).

5. Direktvergleich

Kriterium	EdgeFEM	Palace	Gewinner
Architektur-Fit (Python-SDK, Script-Pipeline)	Bibliothek im selben Python-Skript wie heute openEMS	externes MPI-Binary, WSL/Docker nötig	EdgeFEM
Windows-Story (PyInstaller-GUI für Enduser)	nur ein Windows-Wheel in unserer CI nötig	kein offizielles Windows, WSL „best-effort“	EdgeFEM
Ergebnis-Kompatibilität	direkt Touchstone .sNp + .vtu → existierende Viewer	CSV-Adapter + VTK nötig	EdgeFEM
Integrationsaufwand	niedrig (Script-Template + Exporte)	hoch (Config-Gen, WSL-Runner, CSV-Adapter, MPI-Lifecycle)	EdgeFEM
Port-Abdeckung	Lumped ja, Hohlleiter-WavePort eingeschränkt, MSL/TEM nein	Lumped R/L/C, WavePort mit Dielektrikum-Querschnitt, Floquet	Palace
Physik-Abdeckung	dispersiv ja; nur Frequenzbereich; ab ca. 1 GHz	Eigenmoden, Transient, adaptiver Sweep, AMR; keine dispersiven Medien	geteilt
Reife und Support	7 Sterne, Einzelautor, v1.0, CI-Benchmarks, ehrliche Limits	600 Sterne, AWS-Team, 3515 Commits, HPC/GPU, dicke Doku	Palace
Leistung / Problemgröße	single-threaded, Direktlöser, bis ca. 1 M DOF, kein AMR	MPI, GPU, AMR, große Strukturen	Palace
Lizenz	MIT	Apache-2.0	neutral (beide ok)

Leseart: EdgeFEM gewinnt jedes Integrationskriterium, Palace gewinnt jedes Solver-Qualitätskriterium. Für die Entscheidung ist deshalb maßgeblich, ob man ein schnelles, Windows-freundliches erstes FEM-Backend oder den langfristig leistungsfähigeren Solver suchen will.

6. Empfehlung

> **Empfehlung: EdgeFEM als erstes FEM-Backend (MVP); Palace als optionales schweres Backend, sobald dessen Fähigkeiten gebraucht werden — gebaut auf einer solver-neutralen Abstraktion (Phase 0) und einer gemeinsamen gmsh-Meshpipeline (Phase 1).**

6.1 Begründung

- **Windows-Verteilung ist entscheidend** für eine per GitHub-Release ausgelieferte PyInstaller-App: EdgeFEM kostet eine Wheel-Build-Pipeline; Palace erzeugt bei jedem Nutzer WSL/Docker-Fallstricke (Pfad-Mapping, Ressourcen-Limits, Setup).
- **Der Integrationsaufwand halbiert sich:** Touchstone-Export heißt, S-Parameter-Viewer, Marker, Delta-Modus und JSON-Persistenz (in v0.10.0 gebaut) laufen sofort weiter.
- **Dispersive Materialien** sind der größte physikalische Abstand zwischen FDTD und jedem FEM — EdgeFEM schließt ihn (FR4-Modelle), Palace nicht.
- **Das Meshing-Problem (Phase 1) ist bei beiden identisch** — die teuerste Investition geht in keine Richtung verloren.
- **Wann Palace:** Eigenmoden/Q-Faktor, Strukturen jenseits 1 M DOF, MPI-Parallelität oder exakte Mikrostriplen-Wellenporten — dann als zweites Backend hinter derselben Seam.

6.2 MVP-Scope unter EdgeFEM

Pflicht im MVP	Bewusst eingeschränkt / später
Lumped-Ports (R/L/C) — PortDef liefert alles bereits	MSL-Port als Lumped am Port-Box-Ende modellieren (Doku-Hinweis)
S-Parameter: ein Port pro Lauf, multirun.py-Muster nutzbar	WavePort nur für homogene Hohlleiter
Betrieb ab ca. 1 GHz (2.4/5.6-GHz-Beispiele passen)	Time-Domain, Eigenmoden, AMR, große Arrays → Palace/HFSS
Touchstone- und VTK-Ergebnisse (Viewer unverändert)	Farfield-Adapter und Fortschritts-Parser (kleine Arbeiten)

7. Integrationsarchitektur (solver-neutral)

7.1 Backend-Schnittstelle (Phase 0)

```
class SolverBackend:
    id: str                                # "fdtd" | "fem"
    def prepare(project, work_dir) -> RunPlan
    def command(run_plan) -> list[str]     # Popen-Argv
    def feed_progress(line) -> ProgressState
    def load_results(project) -> ResultBundle
```

- *fdtd* = Wrap des heutigen Verhaltens (SCRIPT_TEMPLATE + EngineProgress), unverändertes Verhalten, alle 434 Tests grün.
- *fem* = neu; lädt entweder das EdgeFEM-Skripttemplate (Python in-process) oder über spätere Erweiterung palace (Config + WSL-Lauf).
- Projektmodell: neuer Top-Level-Key *solver* mit Backend-Wahl und FEM-Parametern (np, Dichte/Frequenzliste, Refinement) — rückwärtskompatibel.

7.2 Gemeinsame Meshpipeline (Phase 1, identisch für beide)

```
Project.primitives + CAD-Tessellierung (freecad_mesh.py / stl_import.py)
        |
        v
gmsh-Python-SDK (OCC)  ->  physical groups:
        Volumen-Attribute = Materialien, Flächen = PEC/PMC/Absorbing/Ports
```

```
        PortDef start/stop -> Port-Rechtecke fragmentieren
        |
        v
.msh (für EdgeFEM: Format msh2; für Palace: MFEM-lesbar) + Attribut-Tabelle
```

- Zero-Thickness-Metal (CSX-Prioritätenmodell) → Innenflächen-PEC / Conductivity-Sheets.
- MeshSpec.cells_per_wavelength → gmsh-Meshgröße; Palaces Refinement/AMR als zweite Stufe.
- Alternative für einen frühen Prototyp: strukturiertes Hex-Box-Gitter (Palace unterstützt Hex; EdgeFEM nicht dokumentiert).

7.3 Ergebnis-Adapter (Ziel: Viewer unverändert)

Quelle	Ziel-Format	Betroffener Viewer
EdgeFEM export_touchstone()	sparams/.s1p/.s2p (+ sparams.json)	sparam_viewer — ohne Änderung
EdgeFEM export_fields_vtk() / Palace paraview/	VTK-Serie (.vtu/.vtr, PVD)	vtk_viewer (VTU-Reader prüfen)
EdgeFEM pattern-CSV / Palace farfield-rE.csv	nf2ff.json-Adapter	antenna_viewer — ohne Änderung
Palace eig.csv	neue Eigenmoden-Ansicht	neu
Engine-/Solver-Log	je Backend eigener Progress-Parser	Statusleiste

8. Phasenplan

Phase	Inhalt	Aufwand	Bemerkung
0	Solver-Abstraktion: SolverBackend, Project.solver-Key, Runner-Argv, fdtd-Backend hinter die Seam	2–3 Tage	neutral, Voraussetzung für alles
1	Mesh-Pipeline: Geometrie → gmsh → physical groups, Port-Rechtecke, Attribut-Tabelle, msh2/MFEM-Export	4–6 Tage	größtes Risiko zuerst, solver-neutral
2	Input-Erzeugung	3–4 Tage	EdgeFEM: project_run.py-Template; Palace: JSON-Config-Generator
3b	Windows-Wheel für EdgeFEM (GitHub Actions, MSVC + Eigen + gmsh)	2–3 Tage	nur EdgeFEM nötig; Selbsttest im Settings-Dialog
3	Runner/Progress/Abort	2–3 Tage	EdgeFEM: fast entfalls (bestehender Runner); Palace: WSL-Lifecycle
4	Results-Adapters (Touchstone, VTK, Farfield, Frisch-Logik)	3–4 Tage	EdgeFEM deutlich leichter
5	UI: Solver-Auswahl, FEM-Settings, Inkompatibles sperren, Help-Eintrag	2–3 Tage	MSL-Port bei fem deaktivieren mit Tooltip
6	Tests, Doku (ANLEITUNG_DE/EN), Vergleichsbeispiel FDTD vs. FEM	2–3 Tage	goldene Mesh-/Config-Tests

Gesamt MVP: etwa 3–4 Wochen (Teilzeit). Mit Palace steigen Phase 2–4 deutlich (Config-Generator, WSL-Runner, CSV-Adapter). Nachgeschoben in jedem Fall: Eigenmoden-Viewer, WavePorts, adaptiver Sweep, Farfield-Adapter.

9. Risiken und Mitigationen

Risiko	Betrifft	Mitigation
Reifegang / Bus-Factor 1	EdgeFEM	Version hart pinnen, Source als Submodule/Vendor-Tree mitnehmen (Vorbild: third_party/easyMesh4openEMS), kontrollierte Upstream-Einspielung

Risiko	Betrifft	Mitigation
Windows-Wheel ist eigene CI-Arbeit	EdgeFEM	GitHub-Actions-Workflow windows-latest, Wheel versioniert mitliefern, Selbsttest im Settings-Dialog
Kein TEM-/MSL-Port	EdgeFEM	MVP nur Lumped-Feed; Port-Dialog bei fem mit Erklärungstooltip; langfristig Upstream-PR oder Palace für diese Fälle
Single-threaded, Direktlöser, max. ca. 1 M DOF	EdgeFEM	Frequenzliste dünn wählen, Fortschritt je Frequenzpunkt, Abbruch testen
Instabil unter etwa 1 GHz	EdgeFEM	Backend-Guard: Warnung „bei unter 1 GHz FDTD verwenden“
Gmsh nur msh2	EdgeFEM	Mesh-Export der Pipeline fest auf msh2 setzen
WSL-/Docker-Friktion bei Nutzern	Palace	palace_launcher-Setting mit Selbsttest, Pfad-Mapping, Doku mit Setup-Anleitung
Config-Version-Drift	Palace	Palace-Version pinnen, _resolved.json mitspeichern, Schema-Validierung im Test
Mesh-Qualität / FEM-Konvergenz (dünnen Leiterbahnen)	beide	Port-/Kanten-Meshgröße aus metal_edge_res und cells_per_wavelength ableiten, AMR empfehlen
Ergebnis-Unterschiede FDTD gegen FEM	beide	in der Doku erklären, Vergleichsbeispiel veröffentlichen
Fortschritt und Abbruch (MPI/Kindprozess)	beide	je Backend eigener Parser, palace.json-Polling, Terminate-Verhalten testen
PyInstaller plus Kindprozess (wsl.exe)	beide	winproc-Hidden-Console prüfen, End-to-End-Test in der gebauten EXE

10. Offene Entscheidungen

1. 1. **Primär-Backend:** EdgeFEM (Empfehlung) oder doch Palace, wenn Port- und Reife-Qualität wichtiger sind als Windows-Einfachheit?
2. 2. **Vendoring:** EdgeFEM als git-Submodule mit eigener CI-Build-Pipeline (Empfehlung) oder *pip install edgefem* zur Laufzeit?
3. 3. **MVP-Port-Scope:** nur Lumped (Empfehlung) oder zusätzlich Hohlleiter-WavePort?
4. 4. **Reihenfolge:** Phasen 0 und 1 (Abstraktion + gmsh-Mesh) solver-neutral als Vorratsmaßnahme starten, auch solange die Backend-Entscheidung offen ist? (In jedem Fall die sinnvollste erste Investition.)
5. 5. **Betreibsmodell Palace (falls doch):** WSL2 (Empfehlung) oder Docker — beide über einen einheitlichen launcher-String?

11. Quellen

- AWS Palace — github.com/awslabs/palace (README, Apache-2.0)
- Palace-Dokumentation — awslabs.github.io/palace/stable/ (Installation, Running, Problem Types, Simulation Models, Boundary Conditions, Postprocessing, Configuration File, FAQ: Windows/WSL, Farfield-Feature)
- EdgeFEM — github.com/jman4162/EdgeFEM (README, MIT, Limitations, Roadmap, Validation), jman4162.github.io/EdgeFEM/
- VisualopenEMS-Architektur — Code-Analyse vom 8. November 2026 (app/simulation/{script_gen,runner,progress,multirun}.py, app/models/project.py, app/postproc/*, app/ui/*)
- VisualopenEMS v0.10.0 — github.com/istra711/VisualOpenEMS (releases/tag/v0.10.0)

Erstellt als Planungsunterlage. Es wurden keine Code-, Repo- oder Release-Änderungen vorgenommen.